STUDY_PROTOCOL

STUDY PROTOCOL — EP-AGENT-001

Version: v0.2-DRAFT (2026-10-15) **Status:** DESIGN ONLY — NOT AUTHORIZED FOR EXECUTION — NO RESULTS

This protocol describes how the study **would** be executed after authorization. No step below beyond the harness self-test (§0) may be performed until all conditions in §1 are satisfied.

---

0. Steps permitted before authorization

Only harness checks that do not contact any ExecutionProof endpoint and produce no experimental data:

```
cd /home/ubuntu/EP-AGENT-001
python3 scripts/validation/validate_fixtures.py
python3 scripts/validation/validate_schemas.py
python3 scripts/validation/harness_selftest.py
```

Their output is **harness self-test evidence only** — it shows that fixtures load and normalizers emit schema-complete contracts. It is never reported as an experimental result.

1. Pre-execution checklist (all required; record evidence for each)

#	Condition	Evidence
1	Preregistration freeze checklist items 1–14 complete (`PREREGISTRATION.md` §16)	Completed checklist with evidence (written; no digital signature)
2	`PREREGISTRATION.md` SHA-256 computed and recorded	Hash value + timestamp
3	`MANIFEST.sha256` regenerated at freeze and verified (`sha256sum -c`)	Verification output
4	10-question gate re-run on the frozen design; design-scope verdict GO or GO WITH BOUNDARY and Q8, Q9 PASSED	Gate record
5	Audit refresh check: no new UD/UW brief received since the 2026-10-15 audit that changes scope (Rule 10)	Note
6	Exposure log reconciled (`UNIVERSITY_EXPOSURE_LOG.md`)	Log entries
7	Derek Hone’s explicit execution authorization received in writing, citing the preregistration SHA-256	Authorization text
8	Environment matches `ENVIRONMENT.md`; `check_versions.py` all PASS	Script output
9	ExecutionProof sandbox reachable at the pinned version; fixture policy loaded and its configuration hash matches the frozen value	Hash comparison
10		Env dump (secrets-free)

#	Condition	Evidence
	No production credentials present in the environment; downstream targets are mock only	
11	`results/` and `proofrecords/` contain only their placeholder files	Directory listing

If any item fails, execution does not start. Record the failure as a pre-execution deviation (§10).

2. Framework setup (per framework, each in its own clean virtual environment)

Each framework is installed in a separate venv to avoid cross-framework dependency interference and to make dependency burden (S3) measurable.

Step	OpenAI Agents SDK	LangGraph	CrewAI	MAF
venv	`.venv-openai_agents`	`.venv-langgraph`	`.venv-crewai`	`.venv-maf`
install	`pip install openai-agents==0.24.0`	`pip install langgraph==1.2.15`	`pip install crewai==1.15.28`	`pip install agent-framework==1.22.0`
version check	`check_versions.py --framework openai_agents`	`... langgraph`	`... crewai`	`... maf`
single-agent config	One `Agent`, one function tool; **no handoffs** configured	One graph with one agent node + one `ToolNode`; no sub-agents	One `Agent` (`allow_delegation=False`), one `Task`, one `BaseTool`	One agent, one function tool, one `FunctionMiddleware`
model dependence	Tool invoked deterministically (forced tool choice or direct runner invocation) so that LLM variability does not affect which tool/ arguments are submitted	Deterministic graph route to `ToolNode` with fixed tool-call message	Deterministic tool invocation for the fixed task	Deterministic function invocation through middleware

Setup timings for S2 start at venv creation and stop at the first successful `/verify` response in that venv, using a non-study smoke fixture (not B1–B5).

Determinism rule. Tool arguments are populated directly from the fixture, not generated by an LLM. Where a framework requires a model in the loop, a deterministic stub model or forced tool choice is used and documented. This isolates the integration path (the study object) from model behaviour (not studied).

3. Fixture loading procedure

1. Verify each fixture's SHA-256 against the frozen values in `MANIFEST.sha256`. Any mismatch → abort (INCONCLUSIVE candidate: corrupted artifacts).
2. Load fixture JSON; validate with `validate_fixtures.py` logic.
3. Load the fixture's `fixture_policy_context` into the ExecutionProof sandbox via the policy endpoints, **once**, before any case runs; record the resulting configuration hash.
4. Pass `fixture.contract` fields to the adapter as framework-native tool arguments.
5. The adapter's normalizer produces the Request Contract; the harness stores the normalized contract verbatim before submission.

4. Case execution order

- Framework order: randomized once at freeze with a recorded seed; the same order is used for every case.
- Case order within a framework: B1, B2, B3, B4, B5.
- Each cell (f, c) runs: 1 decision call (primary endpoints) → 5 warmup latency calls (discarded) → 30 measured latency calls.
- No cell is re-run except under §10 deviation rules. Every attempt is logged, including failed ones.

5. Data capture requirements (per decision call)

Field	**Source**
`framework_id`, `framework_version`	adapter / installed package metadata
`case_id`, `fixture_sha256`	fixture
`normalized_contract` (full JSON)	normalizer output
`schema_valid`	frozen schema check
`verify_request_sent_at`, `verify_response_received_at`	`time.perf_counter_ns` + UTC ISO 8601
`http_status`, raw `/verify` response body	HTTP client
`observed_verdict`	response
`proof_record_id`	response
`proof_record_resolves`	`/proof/{id}` GET result
`framework_native_result` (serialized)	framework runtime output
`verdict_in_native_result`, `proof_ref_in_native_result`	string-equality checks
`manual_intervention`	boolean (must be false for `integration_complete`)
`core_modification`	boolean (must be false)
`notes`, `deviation_ids`	operator

Capture is append-only JSONL. Files are written only under `results/` after authorization.

6. ProofRecord capture procedure

1. Read `proof_record_id` from the `/verify` response.
2. GET `/proof/{id}`; store the full response under `proofrecords/<framework>/<case>.json`.
3. Record the SHA-256 of the stored body.
4. Confirm the ProofRecord's verdict field equals the `/verify` verdict and its request reference equals the submitted `request_id`.
5. Confirm the same `proof_record_id` appears unchanged in the framework-native tool result.

No ProofRecord is modified, re-signed, or replayed. Integrity-attack testing is UD-reserved and is not performed.

7. Latency measurement protocol (RF internal measurement)

Parameter	**Value**
Timer	`time.perf_counter_ns()`
Warmup	5 calls per cell, discarded
Sample	30 measured calls per cell (600 total)
Boundary S4 (normalization)	entry → exit of `normalize_to_request_contract`

Parameter	Value
Boundary S5 (adapter round trip)	adapter receives framework tool invocation → verdict returned to framework
Concurrency	serial; no parallel calls; exactly one tool call per run (no single-agent parallel tool calls)
Inter-call gap	none beyond natural loop overhead
Environment	same host, same session, no other heavy workloads; host spec recorded
Reporting	median, p95, min, max, raw values; optional bootstrap 95% CI for median (1,000 resamples, frozen seed)

Latency calls reuse the same fixture contents. Each call produces a fresh `request_id`, `nonce`, and `requested_at` by normal operation; latency calls are **not** a replay test and their verdicts are recorded only as a consistency check, not as additional primary-endpoint cells.

8. Data preservation

- All raw JSONL, ProofRecords, logs, environment dumps, and `pip freeze` outputs are preserved.
- After the run, compute SHA-256 for every file in `results/` and `proofrecords/` into `results/RESULTS_MANIFEST.sha256`.
- No file is deleted or edited after creation. Corrections are appended as new, dated records (Rule 11).
- A FAIL and any later remediation are preserved as two separate facts.

9. Mid-run integrity

- If a framework crashes, record the traceback; do not patch the framework mid-run.
- If the ExecutionProof sandbox becomes unreachable, stop, record, and evaluate against the INCONCLUSIVE conditions in `PASS_FAIL_RULES.md`.
- No ExecutionProof core change is permitted during the run. If one appears necessary, stop: P5 = true is recorded.

10. Deviation handling

Type	Handling
Pre-execution (e.g., framework replacement, version bump)	Written amendment, new CHANGELOG entry, regenerated manifest, new preregistration SHA, gate re-run
During execution	Log with ID, timestamp, description, affected cells; do not alter classification rules; report in results
Post-execution analysis	Only preregistered analyses are confirmatory; anything else is labelled exploratory

11. Post-execution lockdown

1. Compute and record `RESULTS_MANIFEST.sha256`.
2. Apply `PASS_FAIL_RULES.md` mechanically; record the classification with the evidence for each endpoint.
3. Tag the repository commit.
4. Do **not** publish, archive to Zenodo, or share with university teams until Derek Hone gives a separate publication authorization.
5. Before any university-facing sharing, add an entry to `UNIVERSITY_EXPOSURE_LOG.md`.